IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received 23 June 2026, accepted 18 July 2026, date of publication 24 July 2026, date of current version 7 August 2026.

Digital Object Identifier 10.1109/ACCESS.2026.3716627

RESEARCH ARTICLE

Learning Footstep Constrained Policies Guided by Linear Inverted Pendulum Model for Humanoid Robots

JAEYONG SHIN[1], WOOHYUN CHA[1], AND JAEHEUNG PARK[2,3], (Senior Member, IEEE)

[1]Department of Intelligence and Information, Graduate School of Convergence Science and Technology, Seoul National University, Republic of Korea
[2]Department of Intelligence and Information, Graduate School of Convergence Science and Technology, ASRI, AIIS, Seoul National University, Republic of Korea
[3]Advanced Institute of Convergence Technology (AICT), Suwon-si, Republic of Korea

Corresponding author: Jaeheung Park (park73@snu.ac.kr)

This work was supported in part by the Basic Science Research Program through the National Research Foundation of Korea (NRF) funded by the Ministry of Education under Grant RS-2023-00274280.

ABSTRACT Recent advances in Deep Reinforcement Learning based control policies for legged robots have demonstrated locomotion policies that control robots with unprecedented robustness against uncertainties and disturbances. Most policies are trained to follow velocity and heading commands to navigate the environment, neglecting how real-world environments often impose spatial constraints where only sparse regions of the terrain are allowed for stepping. Conventional RL policies are prone to failure in such footstep-constrained environments, where control policies that can accurately execute foothold commands must be used instead. In this work, we propose using the Linear Inverted Pendulum Model to guide humanoid locomotion policies that can execute foothold commands, enabling locomotion in spatially constrained environments. Especially, an online Linear Inverted Pendulum Model-based center-of-mass trajectory planner guides the locomotion policies to produce stable motion patterns under arbitrary foothold and gait parameter commands. By combining model guidance with learning, our approach reduces the instability and inconsistency common to purely learning-based controllers while simultaneously improving robustness to model mismatch and real world uncertainties that often limit purely model-based methods. Extensive simulation and real-world experiments show that policies trained with the proposed method accurately execute arbitrary foothold and gait parameter commands while exhibiting robustness against model uncertainties and external disturbances.

INDEX TERMS Bipedal locomotion control, bipedal robot, preview control, reinforcement learning.

I. INTRODUCTION

Humanoid robots must rely on well–designed bipedal locomotion strategies to pass through narrow passages and operate in constrained spaces. Recent advances in Deep Reinforcement Learning (DRL) have produced compelling results in this domain. However, much of the literature trains policies to follow base velocity and heading commands [1], [2], [3], often overlooking the crucial task of precise foot placement.

Although prior work has explored tracking arbitrary footstep commands [4], [5], [6], [7], [8], most approaches rely on extensive reward tuning, and even with careful tuning, the intended behavior and gait consistency can be brittle. In particular, the center of mass (CoM) control is often left implicit: CoM trajectories are not explicitly defined or stabilized, leading to inconsistent CoM motion and imprecise touchdowns. In contrast, model-based pipelines explicitly specify desired footholds and optimize the CoM trajectory accordingly, yielding motion patterns that are precise and consistent by construction [9], [10]. Yet, purely model-based methods can degrade under model mismatch and real-world uncertainties—such as unmodeled compliance, friction, or sensing/actuation delays [11], [12]. Moreover, large unmodeled disturbances often force model-based controllers to adjust contact timing or the next foothold, and performing

The associate editor coordinating the review of this manuscript and approving it for publication was Rajesh Kumar.

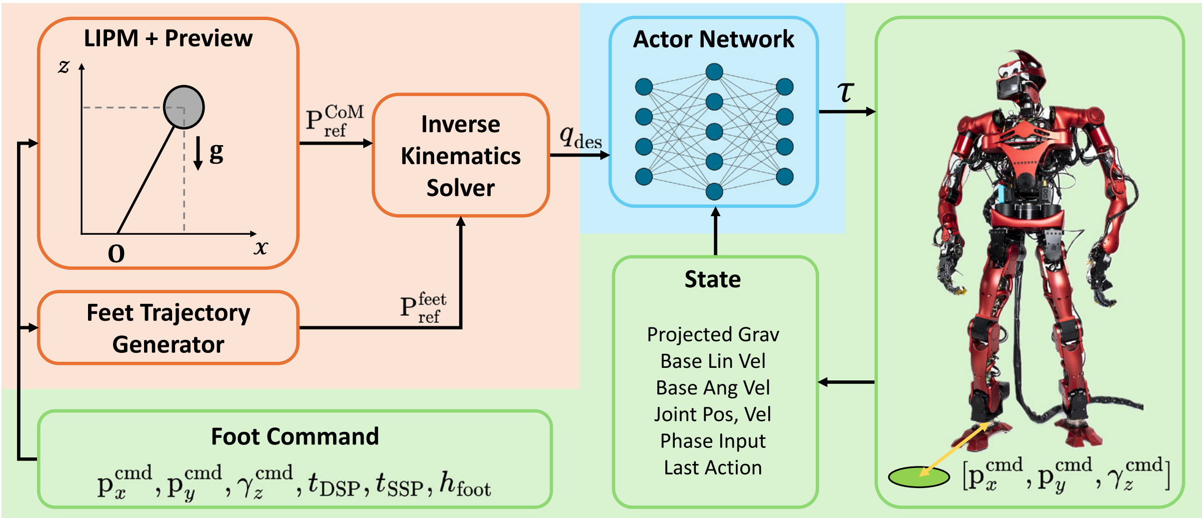


FIGURE 1. Overview of the proposed control framework: The proposed framework integrates the Linear Inverted Pendulum Model (LIPM) with DRL to train humanoid locomotion policies that accurately execute foothold commands. The LIPM is used to generate a reference CoM trajectory that guides the policy toward stable motion patterns. The trained policy outputs joint torques to control the humanoid robot to land its swing foot in the designated configuration.

these updates quickly and reliably can be computationally demanding.

Furthermore, The need for precise stepping is particularly pronounced on heavy, high–payload humanoids. Unlike lightweight platforms that can execute highly dynamic, reactive maneuvers [13], [14], heavy robots typically employ high–reduction gear and carry substantial reflected inertia. As a result, actuator bandwidth is limited, contact transitions induce larger impact forces, and aggressive recovery or rapid re–stepping is risky for both hardware and stability. In such regimes, locomotion should prioritize stability and predictability over agility, making accurate placement of each foothold a pragmatic and often necessary design goal.

To address these issues, we propose a methodology for training humanoid locomotion policies by integrating a reduced-order robot model that captures precise foot placement capabilities and high robustness into a single controller. Our approach leverages the Linear Inverted Pendulum Model (LIPM) [15] to generate online CoM reference trajectories from a sequence of foothold and gait parameter commands. A neural network policy is then trained to track these CoM references, acting as a learned inverse-dynamics controller. The use of LIPM guides the policy to produce more stable and consistent motion patterns than fully reward-based approaches [4], [16] along with precision in command execution, as properly optimized CoM trajectories help satisfy the conditions for the physical feasibility of bipedal locomotion [17]. Although LIPM is prone to yield dynamically sub-optimal trajectories due to model discrepancies, the trained policy learns to control the robot in precise task execution without being overly aligned with the model. Bridging the gap between the simplified LIPM assumptions and real-world dynamics would otherwise require additional components such as whole-body controllers or MPC with explicit contact force regulation. The learning component addresses this gap directly, acquiring robustness to these unmodeled effects through experience rather than explicit modeling.

We validate the proposed method on the full-sized humanoid robot. Experimental results demonstrate that trained policies accurately follow foothold and gait parameter commands while exhibiting consistent motion patterns and strong robustness against model mismatches and external disturbances. The proposed method promises reliable and predictable locomotion policies that can operate in constrained environments common in industrial and outdoor scenarios.

The remainder of the paper is organized as follows. Section II provides a brief overview of previous studies on related topics. Section III illustrates the details of the proposed approach and implementation. Section IV describes the experimental settings and presents the results. Finally, Section V concludes the paper and discusses future research.

II. RELATED WORK

A. LEARNING HUMANOID LOCOMOTION

Humanoid locomotion is challenging due to the inherent instability of bipedal gait and the inertial effects of the upper body. The high number of degrees of freedom (DoF) can cause naive neural policies to produce unnatural or dynamically infeasible gaits that do not transfer well to hardware. To address these issues, previous work has explored extensive reward shaping [1], [18] and imitation learning [3], [19], [20], as well as hierarchical schemes in which high-level modules generate references for low-level controllers [21], [22]. In particular, several approaches adopt reduced-order models to generate reference trajectories that guide low-level RL policies through tracking rewards [21], [22]. However, these methods primarily focus on velocity or gait-level command tracking and do not address precise execution of arbitrary foothold commands in spatially constrained environments.

Beyond model-guided approaches, recent DRL results have shown strong bipedal locomotion conditioned on base-velocity and heading commands [1], [2], [3]. However, such command interfaces do not specify where the feet must be placed. When feasible support regions are sparse (e.g., stepping stones), a policy trained only to track base commands effectively reduces to a blind policy with respect to discrete foothold constraints: it must infer contact decisions implicitly from proprioception and rewards. This makes exploration inefficient, gradients ambiguous, and often yield inconsistent base motion and imprecise touchdowns. We do not claim impossibility, but this setting is demonstrably challenging without explicit foothold information.

The risk is amplified on heavy, high-payload humanoids. High gear reductions and large reflected inertia limit actuator bandwidth and make fast, aggressive recovery or restepping maneuvers undesirable for both hardware longevity and stability. In this regime, relying on a blind velocity-tracking policy to figure out foot placement online can be unsafe: missed or marginal contacts lead to larger impact impulses and reduced stability.

B. LEARNING FOOTSTEP-CONSTRAINED LOCOMOTION

Some previous studies address this issue by training control policies that take foothold commands instead of velocity commands. For instance, [23], [24], [25], [26], [27] present hierarchical control frameworks in which a low-level controller is trained to execute foothold commands from a high-level planner. In particular, [25], [26] use physics models to generate feasible and stable footstep commands. However, these frameworks do not accept arbitrary foothold commands as input.

Other approaches train control policies to handle arbitrary foothold commands, making them independent of specific high-level planners. For example, [16] uses careful reward shaping to guide simulated bipedal characters to follow foothold commands, while [4], [5] adopt similar strategies on real bipedal hardware. However, such fully reward-based approaches are prone to producing inconsistent motion patterns [28]. Our approach focuses on training and deploying control policies that precisely track arbitrary foothold targets while being guided by reference trajectories computed using a reduced-order model of the robot. This approach allows integration with any footstep planner while producing stable and consistent motion patterns, without resorting to complex reward designs.

C. LINEAR INVERTED PENDULUM MODEL AND PREVIEW CONTROL

The Linear Inverted Pendulum Model (LIPM) has been widely used in model-based approaches to generate a walking pattern for bipedal robots [29], [30], approximating the robot dynamics as a point mass on a massless leg. Under the assumption of a constant Center-of-Mass (CoM) height, the relationship between the CoM and the Zero-Moment Point (ZMP), the point on the ground plane where total moments cancel out, becomes:

$$\ddot{x} = \frac{g}{z_c}(x - z_x)$$
$$\ddot{y} = \frac{g}{z_c}(y - z_y) \tag{1}$$

where (x, y) are the XY coordinates of the CoM, (z_x, z_y) the ZMP coordinates, and z_c the constant CoM height. These linearized dynamics allow optimized solutions with low computational cost that are straightforward to implement in planning and control. Given reference ZMP trajectories, one can generate an optimized CoM trajectory through the Preview Control method proposed in [31]. Our implementation of the LIPM-based Preview Control module follows the theory and methodology of [31].

III. METHOD

The proposed method uses the LIPM and simple heuristics to generate reference trajectories for the CoM and feet based on foothold commands and gait parameter commands. The gait parameter command consists of the duration of the gait phases and the peak height of the swing foot. The control policy is trained to follow these reference trajectories and is deployed on real hardware. This section explains the implementation details of our approach, starting with the Reinforcement Learning formulation, followed by command formulation and generation. We then describe how reference trajectories are generated and finally discuss the training process and our sim-to-real transfer strategy.

A. REINFORCEMENT LEARNING

We model the robot locomotion control problem as a discrete-time partially observable Markov Decision Process (POMDP). At each time step t, the agent constructs the observation vector o_t that contains partial information about the system's state s_t, along with a command vector describing the control objective. The agent selects an action $a_t \sim \pi_\theta(\cdot|o_t)$ from its control policy $\pi_\theta(\cdot|o)$. After executing a_t, the environment transitions to $s_{t+1} \sim P(\cdot|s_t, a_t)$, and the agent obtains a reward $r_t = r(s_t, a_t)$. The goal is to maximize the expected discounted return over a horizon T, formulated as $J(\pi_\theta) = \mathbb{E}_{\tau\sim p(\tau|\pi_\theta)}[\sum_{t=0}^{T} \gamma^t r_t]$.

B. FOOTHOLD COMMAND FORMULATION AND GENERATION

We train the robot to execute foothold and gait parameter commands. Let $C = [\mathrm{p}_x^{\mathrm{cmd}}, \mathrm{p}_y^{\mathrm{cmd}}, \gamma_z^{\mathrm{cmd}}, t_{\mathrm{DSP}}, t_{\mathrm{SSP}}, h_{\mathrm{foot}}] \in \mathbb{R}^6$. Here, $[\mathrm{p}_x^{\mathrm{cmd}}, \mathrm{p}_y^{\mathrm{cmd}}, \gamma_z^{\mathrm{cmd}}]$ specify the target foothold position and the yaw angle in the support foot frame, whose X axis aligns with its toe direction and Z axis is vertical. The parameters t_{DSP} and t_{SSP} denote the desired double-support (DSP) and single-support (SSP) phase durations. During SSP, the swing foot moves from its initial configuration to the target of the foot on a cubic trajectory with a peak height h_{foot}.

TABLE 1. Command randomization range.

Parameter	Range	Unit
p_x^{cmd}	$[-0.4, 0.6]$	m
p_y^{cmd}	$[0.2, 0.5]$	m
γ_z^{cmd}	$[-0.1, 0.8]$	rad
t_{DSP}	$[0.0, 0.2]$	s
t_{SSP}	$[0.5, 1.2]$	s
h_{foot}	$[0.05, 0.16]$	m

At the start of each training episode, two randomly sampled commands $[C_1, C_2]$ are stored in the command buffer B_C, where C_1 and C_2 are the current and upcoming command vectors. We track the elapsed time t within the current gait cycle. Upon the end of each gait cycle, i.e. $t > t_{\text{total}} = t_{\text{SSP}} + 2 \cdot t_{\text{DSP}}$, we set $C_1 \leftarrow C_2$, then replace C_2 with a newly sampled command vector. These commands are uniformly sampled from ranges in Table 1. The command ranges are determined by two key kinematic feasibility conditions, as illustrated in Figure 2. First, the p_y^{cmd} bounds ensure that the swing foot is always placed on the outer side of the support foot, preventing leg crossing. Second, the p_x^{cmd} bounds and γ_z^{cmd} bounds are set to avoid singular leg configurations, particularly knee over-extension, while ensuring the commanded foothold remains within the joint limits of the robot.

After updating B_C, the cycle time t is reset to zero. To inform the policy about the gait phase, the current 2D clock value $P(\Phi(t)) = [\cos(\Phi(t)), \sin(\Phi(t))]$ is included in the observation vector, where $\Phi(t) = 2\pi t / t_{\text{total}}$.

C. REFERENCE TRAJECTORY GENERATION

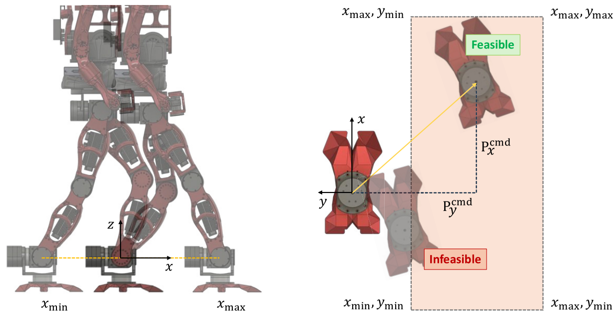
To guide both policy training and deployment, reference CoM and foot trajectories are generated at the policy frequency (125 Hz). At the start of each gait cycle, the command buffer creates a reference ZMP trajectory and a reference pelvis yaw trajectory $B_C = [C_1, C_2]$.

The reference ZMP trajectory, the pelvis yaw trajectory, and the feet trajectory are set within each gait phase φ using cubic-interpolation:

$$A_\varphi(t_\varphi) = \text{cubic}\Big(t_\varphi,\ A_{\varphi,0},\ A_{\varphi,T_\varphi}\Big)$$
$$A \in \Big[z_x,\ z_y,\ \gamma_{\text{des}}^{\text{pelvis}},\ x_{\text{des}}^{\text{feet}},\ y_{\text{des}}^{\text{feet}},\ z_{\text{des}}^{\text{feet}},\ \gamma_{\text{des}}^{\text{feet}}\Big] \quad (2)$$

$t_\varphi \in [0, T_\varphi]$ denotes the current time within the gait phase, and T_φ is the duration of the gait phase φ.

The reference ZMP trajectory is constructed so that it remains in the center of the support foot during the SSP (i.e., $z_{\varphi,0} = z_{\varphi,T_\varphi}$) and is cubic-interpolated between the centers of the previous and next support feet during the DSP. The reference ZMP trajectory is fed into the LIPM-based Preview Control module [31] to compute the reference CoM trajectory $X_{\text{des}}^{\text{CoM}}(t+1) = \big(x_{\text{des}}^{\text{CoM}}\ \ y_{\text{des}}^{\text{CoM}}\ \ z_{\text{des}}^{\text{CoM}}\big)^T$ with $z_{\text{des}}^{\text{CoM}} = 0.68\ [m]$. This value was selected to balance singularity avoidance and feasible command range. CoM heights that are too high or too low narrow the feasible foothold command range due to singular leg configurations, and empirical evaluation on the primary hardware confirmed that the range of approximately 0.58–0.78 m yields stable locomotion, with 0.68 m chosen as a stable operating point. We use a 1.6-second preview horizon following [31], where this duration was shown to provide accurate ZMP trajectory tracking.



(a) Side view (b) Top view

FIGURE 2. Feasible footstep command regions defined in the support foot frame. The shaded region indicates the feasible command space, bounded by kinematic feasibility conditions including leg crossing prevention and singularity avoidance. Commands that would result in foot overlap or singular leg configurations are excluded.

The reference pelvis yaw trajectory is designed to be constant during the DSP (i.e., $\gamma_{\varphi,0,\text{des}}^{\text{pelvis}} = \gamma_{\varphi,T_\varphi,\text{des}}^{\text{pelvis}}$) and cubic-interpolated to the midpoint between the yaw angles of the support foot and the target foothold during the SSP. The desired angle of roll and pitch of the pelvis $[\alpha_{\text{des}}^{\text{pelvis}}, \beta_{\text{des}}^{\text{pelvis}}]$ are always set to zero. Together, we obtain the desired pelvic orientation $\psi_{\text{des}}^{\text{pelvis}}(t+1)$.

The reference foot trajectories follow similar principles: the support foot is designed to maintain its configuration throughout all phases, while the swing foot trajectory is set to maintain its configuration during the DSP and cubic-interpolated to the commanded configuration $[p_x^{\text{cmd}}, p_y^{\text{cmd}}, \gamma_z^{\text{cmd}}]$ during the SSP. The swing foot height peaks at h_{foot} midway through SSP. The desired roll and pitch angles of the feet $[\alpha_{\text{des}}^{\text{feet}}, \beta_{\text{des}}^{\text{feet}}]$ are set to zero. Together, we obtain $[X_{\text{des}}^{\text{feet}}, \psi_{\text{des}}^{\text{feet}}](t+1)$.

Finally, given $[X_{\text{des}}^{\text{CoM}}, \psi_{\text{des}}^{\text{pelvis}}, X_{\text{des}}^{\text{feet}}, \psi_{\text{des}}^{\text{feet}}](t+1)$, an inverse kinematics solver computes the desired lower-body joint positions $q_{des}(t+1)$.

D. TRAINING DETAILS

1) OBSERVATION SPACE

The observation space $\mathcal{O} \subset \mathbb{R}^{65}$ consists of the following:

- $v_{base}(t) \in \mathbb{R}^3$: The base linear velocity.
- $\omega_{base}(t) \in \mathbb{R}^3$: The base angular velocity.
- $g_{proj}(t) \in \mathbb{R}^3$: The base projected gravity vector.
- $q(t) \in \mathbb{R}^{12}$: The lower body joint positions.
- $\dot{q}(t) \in \mathbb{R}^{12}$: The lower body joint velocities.
- $q_{\text{des}}(t+1) \in \mathbb{R}^{12}$: The lower body target joint positions.
- $P(\Phi(t)) \in \mathbb{R}^2$: Current 2D clock value.
- $C_1 \in \mathbb{R}^6$: Command vector for the current gait cycle.
- $a(t-1) \in \mathbb{R}^{12}$: Previously executed action.

TABLE 2. Tracking reward parameters for $r_{tracking}$.

i	w_i	j	$\sigma_{i,j}$
Swing Foot Position	0.2	x	0.15
		y	0.05
		z	0.1
Swing Foot Orientation	0.2	α	0.3
		β	0.3
		γ	0.05
Support Foot Position	0.1	x	0.15
		y	0.15
		z	0.025
Support Foot Orientation	0.1	α	0.3
		β	0.3
		γ	0.3
Support Foot Linear Velocity	0.1	x	0.2
		y	0.2
		z	0.2
Support Foot Angular Velocity	0.1	α	0.5
		β	0.5
		γ	0.5
CoM Position	0.1	x	0.3
		y	0.3
		z	0.05
CoM Orientation	0.1	α	0.05
		β	0.05
		γ	0.05
Target Joint Position	0.4	q	0.5

2) ACTION SPACE

The action space $\mathcal{A} \subset \mathbb{R}^{12}$ is composed of 12 actions, each action component corresponding to a torque command for a lower-body joint. Following [19], which demonstrated that torque–controlled RL-based locomotion yields more compliant behavior than position–controlled approaches, we adopt a torque control reinforcement learning. An additional practical benefit is that torque control obviates manual PD gain tuning, avoiding time-consuming gain scheduling across gaits and operating conditions. The upper-body joints use joint-space PD control to hold their default positions. Each action is bounded in $[-1, 1]$, and the actual torque command is obtained by multiplying the actions with the maximum torques of the corresponding joints.

$$\tau_{\text{input},t} = \tau_{\max} \otimes a_t \, [N \cdot m] \tag{3}$$

Since there is no underlying high-frequency PD controller, the torque-based policy runs at a frequency high enough for optimal control performance and policy reactivity. In this work, 125 Hz is chosen as the policy update frequency.

3) REWARD

The overall reward is defined as:

$$\begin{aligned} r_{total} &= r_{tracking} + r_{energy} + r_{survival} \\ r_{tracking} &= \sum_i [w_i \cdot \exp(-\sum_j ||\delta_{error,i,j}||^2/\sigma_{i,j}^2)] \\ r_{energy} &= -0.0002 \cdot \tau \odot \dot{q} \\ r_{survival} &= 0.1 \end{aligned} \tag{4}$$

TABLE 3. PPO hyperparamters.

Parameter	Value
Horizon (H)	24
Adam learning rate	1×10^{-4}
Number of epochs	5
Number of mini-batches	4
Discount (γ)	0.99
Clipping parameter (ϵ)	0.2
Max gradient norm	1
value loss coefficient	5.0
entropy coefficient	0.001

where $r_{tracking}$ measures how well the robot follows the online reference trajectories, r_{energy} acts as a regularization term and prevents learning infeasible behaviors [32], and $r_{survival}$ is a constant reward for each time step without termination. Table 2 details the parameters for $r_{tracking}$, where the weights w_i reflect the relative importance of each tracking target, and the scaling factors $\sigma_{i,j}$ are tuned to balance foothold precision and generalization to real-world uncertainties. The term $\delta_{error,i,j}$ represents the deviation of the current state i from the desired state from the reference trajectories:

$$\delta_{error,i,j} = (j^i_{\text{des}} - j^i) \tag{5}$$

For the support foot, we set the desired velocity to zero to ensure a rigid stance during the stance phases.

4) TRAINING SETUPS

The policy is trained in the IsaacGym simulator [33], running 4096 parallel environments to collect training samples. These samples are then used to update the policy through the PPO algorithm [34]. Detailed information on PPO hyperparameters can be found in Table 3. Episodes end either when they reach a duration of 10 seconds or when the robot triggers certain termination conditions, such as self-collisions or excessive deviations in pelvis height. The actor and critic networks are Multi-Layer Perceptrons (MLPs), both with two hidden layers of 512 ELU units each. As input, the policy receives a history of observations spanning 10 time steps, sampled with one frame skipped between. The actor network outputs the mean of a Gaussian distribution μ_{actor}, from which actions are drawn as $a \sim \mathcal{N}(\mu_{actor}, \sigma^2_{actor})$. σ_{actor} is also learned during training. Additionally, the *Mirror Symmetry Loss* from [35] is incorporated to encourage symmetrical gait patterns.

5) SIM-TO-REAL TRANSFER

For sim-to-real transfer, we apply the state-dependent joint torque space perturbation method from [36], which has been validated to provide robustness against realistic uncertainties including sensor noise, actuator variability, and unmodeled dynamics without requiring explicit domain randomization. The maximum torque perturbation injected into the joints is set to 50 Nm, implicitly accounting for actuator degradation and unmodeled joint dynamics, while the maximum force perturbation to the base link is set to 80 N. In addition,

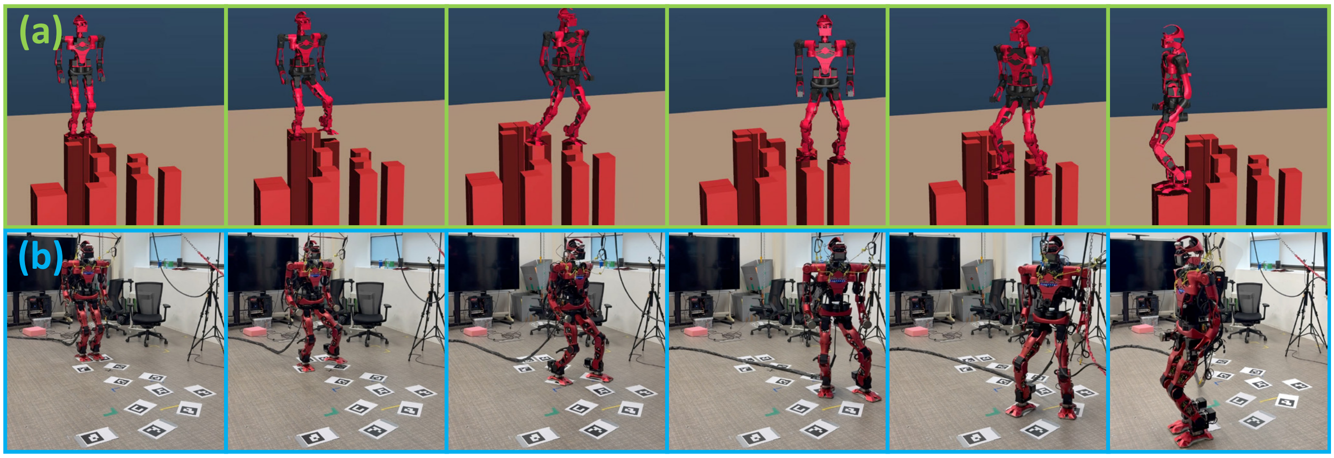


FIGURE 3. Snapshots of the trained policy executing the sequence of footstep commands in simulation (a) and on real hardware (b): Using these commands, the policy successfully drives the robot across stepping stones, under the assumption that stone positions are known. The same scenario is also tested in simulation with an additional 30 kg of mass added to the base link, and the policy maintains the same performance, showcasing its robustness to modeling uncertainties. Meanwhile, real-world trials confirm seamless sim-to-real transfer and precise footstep execution. In the real-robot snapshots, ArUco marker depict the stepping stones. These experiments highlight the potential of the trained policy in traversing spatially constrained environments.

the action delay is uniformly sampled from $[0.0, 0.01]$ s during training to emulate system latencies and sensor noise effects.

To boost robustness against external disturbances, every 4 s we apply an external force to the robot's base link in the XY plane, with magnitude uniformly sampled from $[0, 500]$ N and duration uniformly sampled from $[0.0, 0.2]$ s. This simulates random external pushes, prompting the policy to learn to override commands and focus on stabilizing the robot with various balancing strategies when subjected to disturbances.

Finally, the Lipschitz regularization loss $\mathcal{L}_{smooth}$ from [37] is employed to prevent vibrant motions in real robot deployment.

We combine these terms into a total loss function, presented as follows:

$$\begin{aligned} \mathcal{L}_{total} &= \mathcal{L}_{PPO} + w_{mirror} \cdot \mathcal{L}_{mirror} + w_{smooth} \cdot \mathcal{L}_{smooth} \\ w_{mirror} &= 4.0,\ w_{smooth} = 0.003 \end{aligned} \tag{6}$$

E. COMPUTATIONAL COST AND REAL-TIME FEASIBILITY

The proposed method involves two main online computations at each policy step. The first is the LIPM-based reference generation and Inverse Kinematics (IK) module. This module generates the CoM trajectory from the reference ZMP trajectory using preview control and then computes the target joint positions by solving inverse kinematics with the generated CoM trajectory and the reference foot trajectory. The second is the neural network forward pass. In our implementation, the preview-control and IK module takes approximately $5 - 10$ μs, while the neural network forward pass takes approximately $50 - 100$ μs per step. The total online computation time is therefore approximately $55 - 110$ μs, corresponding to only 0.7–1.4% of the 8 ms control period imposed by the 125 Hz policy frequency. This confirms that the proposed controller can be executed in real time with a large computational margin.

Moreover, the controller-side computational cost is independent of the internal structure of the upstream planner, as the policy only receives low-dimensional commands such as the desired foothold and gait parameters. Any planner that outputs these commands can, therefore, be coupled to the controller without modifying the policy. When nominal gait parameters are fixed, integration becomes even simpler as the planner only needs to provide the desired foothold command at each gait cycle.

IV. EXPERIMENTAL RESULTS

In this section, we demonstrate the effectiveness of our proposed approach through both simulation and hardware experiments and compare with two baselines. The first baseline replaces the LIPM-generated CoM trajectory with a heuristic one, leaving the rest of the pipeline unchanged, while the second follows the periodic reward–based method introduced in prior work [4], [5]. We chose these baselines to (i) isolate the specific contribution of the LIPM preview via a drop-in heuristic CoM planner and (ii) benchmark against a representative reward-shaped footstep controller without model guidance, thereby bracketing the design space from model-guided to fully reward-driven approaches. Simulation studies are conducted in MuJoCo [38] and real-world validation is performed on the humanoid platform.

1) HEURISTIC COM REFERENCE TRAJECTORY

This ablation removes the LIPM preview controller and substitutes a phase-based heuristic for the CoM reference [39]. During the first **DSP**, the CoM is held at the midpoint between the two supporting feet. During **SSP**, the CoM moves to the midpoint between the center of the foot stance and next

TABLE 4. Results of simulation and real robot experiment.

(a) Results of Arbitrary Commands Experiment			
Methods	Max Error δ_{XY}	Mean μ	Std σ
Reward based	0.3725	0.0534	0.0692
Heuristic CoM	0.0736	0.0276	0.0127
Proposed Method	**0.0571**	**0.0221**	**0.0118**
(b) Results of Simulated Stepping Stones Experiment			
Methods	Success Step	Mean error δ_{XY}	Mean error δ_{γ}
Reward based	0	-	-
Heuristic CoM	9	0.0359	0.0194
Proposed Method	**12**	**0.0133**	**0.0065**
(c) Results of Real Robot Stepping Stones with Yaw Experiment			
Methods	Max Error δ_{XY}	Mean μ	Std σ
Heuristic CoM	0.1047	0.0590	0.0244
Proposed Method	**0.0229**	**0.0101**	**0.0072**
Methods	Max Error δ_{γ}	Mean μ	Std σ
Heuristic CoM	0.0672	0.0293	0.0083
Proposed Method	**0.0294**	**0.0130**	**0.0084**

swing-foot target. During the second **DSP**, it holds at the midpoint of the future double support configuration. Each phase is time-parameterized with cubic interpolation and zero end velocities. The yaw reference follows the same design as in the proposed method, and the CoM height is fixed. Given this CoM reference and the swing-foot trajectory, we solve the inverse kinematics to obtain q_{des}; the observation and reward structures are identical to those used in the proposed method.

2) PERIODIC REWARD-BASED POLICY

We reproduce the periodic reward-based policy of [4], which shapes rewards for accuracy in foot placement. We match the observation and reward definitions as closely as our environment allows and train under the same command ranges, action space, and training settings. Although [4] was demonstrated on a smaller and lighter bipedal platform than ours, the original work reported foothold position errors of approximately 0.06–0.1 m, and our reproduction achieves comparable accuracy, confirming the faithfulness of the reproduction.

A. SIMULATION RESULTS

1) EXECUTING ARBITRARY COMMANDS

In this experiment, the trained policy is evaluated on a sequence of 30 randomly sampled foothold and gait parameter commands. These commands are drawn from the parameter ranges listed in Table 1 and are then provided sequentially in the test environment. At the end of each gait cycle, we measure the difference between the desired foothold command and the actual configuration of the swing foot. The errors are calculated by the Euclidean distance between the target foothold and the current foothold in meters. Table 4 (a) summarizes the resulting errors, showing that the proposed method both resulted on lowest maximum error and the mean error. Moreover, Figure 4 reports the ground reaction forces (GRFs) during the step command sequence for all methods. The proposed method yields the smoothest and most bounded vertical GRF profiles with regular left–right alternation, indicating stable contact transitions. In contrast, the two baselines exhibit intermittent force spikes around step transitions, resulting in a less stable step and a poorer impact moderation.

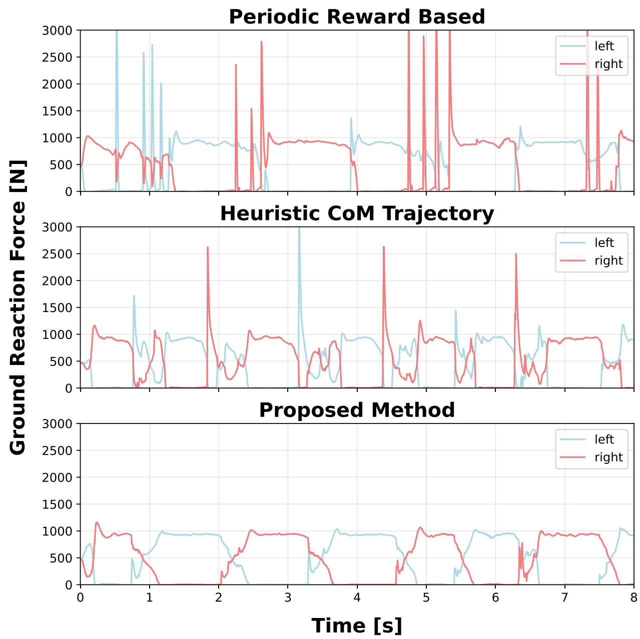


FIGURE 4. Results of ground reaction force while executing arbitrary foot commands.

2) STEPPING STONES TRAVERSAL

In this experiment, the trained policy directs the robot to traverse stepping stones while ensuring high-precision foothold placement to avoid falling. At the end of each gait cycle, we measure the foothold error with the Euclidean distance. The statistics on the measured errors are shown in Table 4 (b). Because we have not yet developed a high-level planner, the coordinates of the stepping stones in the global frame are provided directly to the controller. At the start of each gait cycle, the controller estimates the distance between the current configuration of the support foot and the location of the target stepping stone to generate the command vector. In this way, if a step undershoots or overshoots, the next command is adjusted from the current stance to steer the swing foot back to the stone center. The stepping frequency is fixed at 1 Hz ($t_{\text{DSP}} = 0.2$ s, $t_{\text{SSP}} = 0.8$ s), with a foot lift of $h_{\text{foot}} = 0.08$ m. Figure 3 (a) shows snapshots from simulation, where each stone measures 0.2×0.2 m, while the foot dimensions of the robot are 0.3×0.15 m. Result in Table 4 (b) shows that the periodic reward-based policy failed within the first step, the heuristic CoM policy fell after nine steps, and only the proposed method completed the entire sequence. It also achieved the smallest foothold

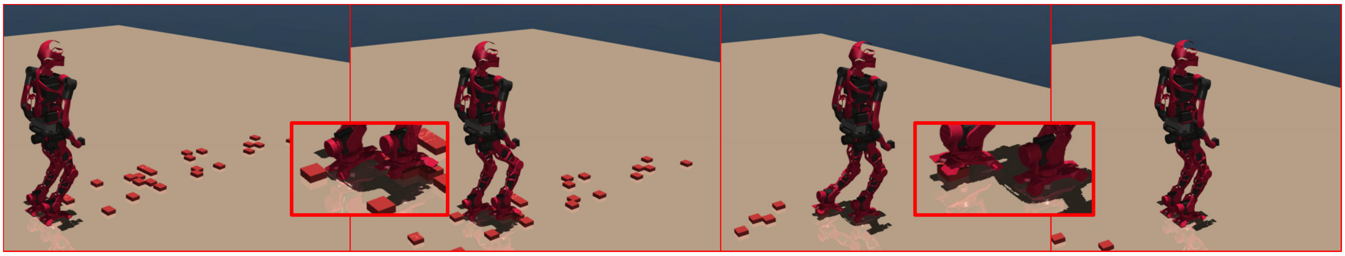


FIGURE 5. Snapshots of the trained policy traversing across a terrain scattered with multiple obstacles: The policy shows robust performance against unstable foot contacts, foot rolling, and variations in terrain elevation, effectively handling scenarios where underlying LIPM assumptions break down. As disturbances from the obstacles grow too large, the controller automatically overrides incoming commands and initiates high-frequency stepping strategies to preserve balance.

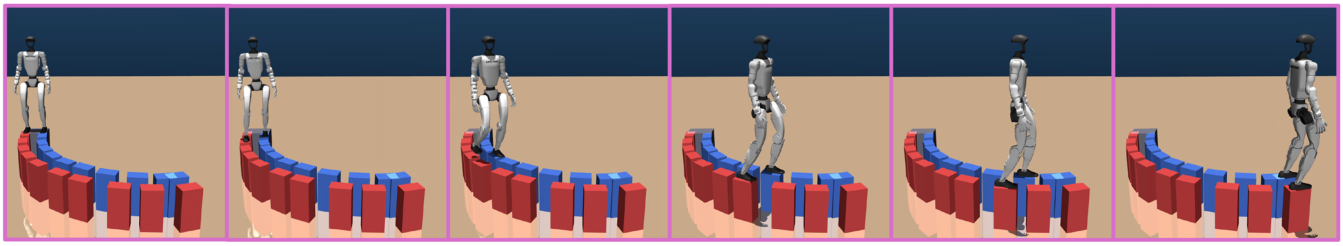


FIGURE 6. Snapshots of the proposed policy executing arbitrary footstep commands on the Unitree G1 humanoid in simulation: Despite the G1 having a different morphology, mass distribution, and joint configuration from the primary hardware platform, the policy trained with the proposed framework accurately executes the commanded footholds while maintaining stable motion patterns. This demonstrates the cross-platform generalizability of the proposed method without requiring modifications to the reward structure, observation space, or sim-to-real transfer strategy.

errors, confirming that it places the feet more accurately and maintains the most stable stepping on the stepping stone course. Overall, these results demonstrate that, given a suitable high-level footstep planner or manually specified foothold locations, the proposed policy can guide the robot through spatially restricted environments with accurate foot placement.

3) TESTING ROBUSTNESS

In this experiment, the trained policy is evaluated under conditions with model uncertainty, violations of underlying LIPM assumptions, and external disturbances. The first scenario adds 30 kg to the mass of the base link of the robot to simulate a significant mass mismatch. Even with this extra load, the robot traverses the same stepping stones shown in Figure 3(a) with no particular performance degradation. In the second scenario, obstacles with heights of 4 cm are scattered across the terrain, causing unstable contact, foot rolling, and unexpected changes in terrain height. These break the assumptions of the LIPM. They also hinder strict adherence to the designated foot trajectories, given that roll and pitch commands are set to zero. As shown in Figure 5, the policy remains capable of executing foothold and gait pattern commands under these uncertainties. When disturbances exceed a tolerable threshold, the robot autonomously overrides commands, adopting high-frequency stepping to recover balance. These results highlight the ability of the policy to balance precise reference tracking with robust stabilization.

To quantitatively assess robustness to external disturbances, we conduct a large-scale push recovery experiment across 10,000 parallel environments in IsaacGym. In each environment, the policy executes randomly sampled foothold and gait parameter commands drawn from the training ranges over a 10-second episode. At $t = 5$ s, an external force of randomly sampled magnitude $F \in [0, 500]$ N is applied to the base link in the XY plane for 0.2 s. The success rate is measured as the proportion of environments in which the robot does not fall until the end of the episode. The contact force is measured by taking the maximum foot contact force within each environment and then averaging these values across all successful environments. The same experiment is conducted for the periodic reward-based policy and the heuristic CoM baseline for comparison.

TABLE 5. Results of push recovery experiment (10,000 environments).

Method	Success Rate	Max Ground Reaction Force [N]	
		Mean μ	Std σ
Reward-based	85.96%	14535.38	3090.65
Heuristic CoM	99.78%	13517.81	4056.81
Proposed Method	**99.80%**	**5319.71**	1987.10

Table 5 summarizes the results. The proposed method and the heuristic CoM baseline achieve comparable success rates, while the reward-based baseline falls significantly behind. However, a clear distinction emerges in the contact force behavior: the proposed method generates a mean maximum contact force roughly half that of both baselines. This demonstrates that the proposed method achieves equivalent robustness to external disturbances while producing substantially smaller impact forces, reflecting the smoother and more stable contact behavior induced by the LIPM-guided CoM

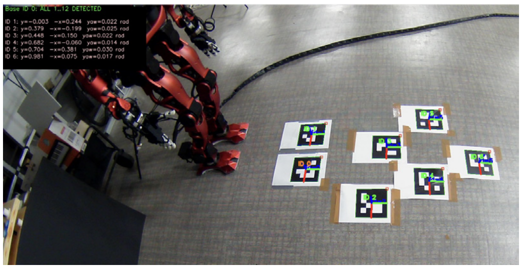


FIGURE 7. Foothold targets from ArUco markers. An external camera detects the markers and estimates their poses and orientations; the marker with ID 0 defines the global reference frame. Poses of all other markers are expressed relative to the ID 0 frame and then converted into foothold commands for the controller.

reference. This property is particularly important for heavy humanoid platforms, where large impact forces pose risks to both hardware integrity and stability.

4) CROSS-PLATFORM GENERALIZABILITY ON UNITREE G1

To evaluate the transferability of the proposed framework across different robot morphologies, we train and evaluate the policy on the Unitree G1 humanoid robot in simulation. The Unitree G1 has a different morphology, mass distribution, and joint configuration from the primary hardware platform, making it a suitable candidate for assessing cross-platform generalizability. Unlike the primary platform which uses torque control at 125 Hz, the G1 policy is trained with position control at 50 Hz, reflecting the different control interface of the platform. The training pipeline otherwise remains identical to that described in Section III, with no modifications to the reward structure, observation space, or sim-to-real transfer strategy. The command ranges are adjusted to reflect the kinematic limits of the G1 platform.

As shown in Figure 6, the policy successfully executes a sequence of arbitrary foothold and gait parameter commands on the G1 platform in simulation, achieving accurate foot placement and stable motion patterns. This indicates that the proposed framework is not specific to a single robot morphology and generalizes across different humanoid platforms.

B. REAL ROBOT RESULTS

In this section, the proposed method and the heuristic CoM trajectory policy are zero-shot deployed on real hardware. As shown in the previous simulation results, the periodic reward-based policy produced frequent peaks in the vertical ground reaction force, although it included an explicit penalty for forces exceeding a threshold during training. Since these peaks exceeded our hardware safety envelope, we judged the policy unsafe and did not deploy it on the real robot. In real robot, we measure the foothold error as in Section IV-A4.

1) STEPPING STONES WITH YAW COMMANDS

Figure 3 (b) presents snapshots of the proposed policy executing a sequence of footstep commands on real hardware, where the policy receives foothold commands from ArUco markers detected by an external camera. Figure 7 shows the camera view used to estimate the marker poses. The marker with ID 0 is defined as the *global* frame, and each detected marker pose is first estimated in the camera frame and then expressed relative to the ID 0 marker. The resulting poses are remapped into the controller command space and sent online to the controller, as in Section IV-A4. Steps 7 and 9 share the same foothold position but require different yaw commands, since two ArUco markers cannot be placed at the same location (overlapping markers are not reliably detected by the camera), we reused the step-7 position and specified the step-9 yaw explicitly in the command.

As summarized in Table 4 (c), which reports foothold error on hardware, the proposed method achieves substantially higher accuracy and more stable stepping than the heuristic CoM baseline. Qualitatively, the accompanying video shows that the heuristic baseline exhibits more abrupt high-impact contacts, whereas the proposed method produces smoother transitions and a noticeably more stable gait.

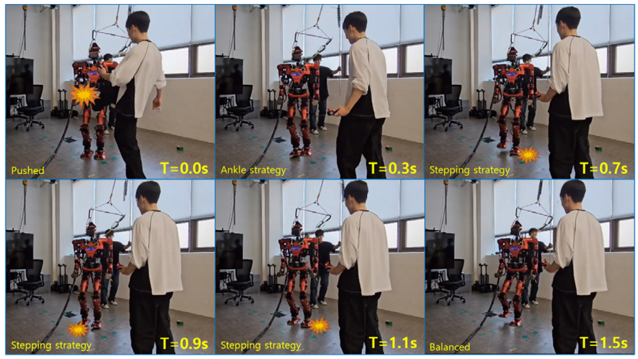


FIGURE 8. Snapshots of the robot countering an external push: Although the policy normally follows gait parameter commands, including stepping frequency, when disturbances become too large, it overrides these instructions and switches to high-frequency stepping to maintain balance.

2) TESTING ROBUSTNESS

In a separate experiment, the robot is commanded to perform 1 Hz stepping in place while being subjected to external pushes. As shown in Figure 8, when disturbances become significant, the robot automatically overrides the footstep and gait parameter commands, instead adopting the high-frequency stepping strategy to maintain balance. This demonstrates that even without online constrained optimization, the proposed LIPM-guided learning-based controller withstands substantial disturbances and maintains balance.

To further examine the disturbance response in detail, we conduct an additional experiment on the real robot in which it executes forward stepping commands of 0.2 m at a fixed stepping frequency of 1.25 Hz ($t_{\text{total}} = 0.8$ s) while an external push is applied along the lateral Y direction. Figure 9 shows the resulting desired and actual CoM trajectories along the Y-axis together with the measured push force. At the moment of the push, the actual CoM

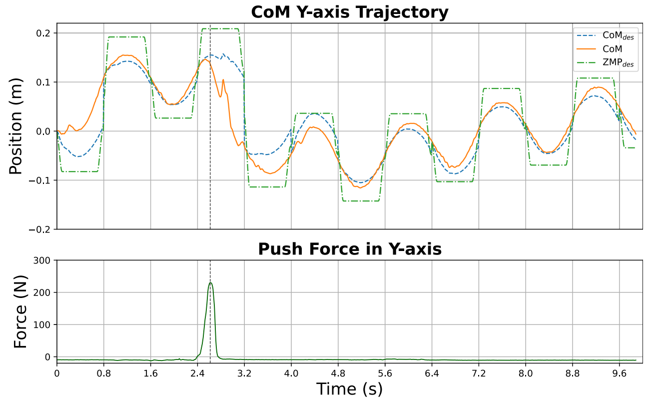


FIGURE 9. CoM tracking under an external push on the real robot: The robot executes forward stepping commands at 1.25 Hz while an external push is applied in the lateral (*Y*) direction. (Top) Desired and actual CoM trajectories along the *Y*-axis. (Bottom) Measured push force. Following the push, the actual CoM deviates from the desired trajectory, and the policy temporarily overrides the commands to recover balance before rapidly converging back to accurate tracking.

deviates significantly from the desired trajectory. In response, the policy temporarily overrides the incoming commands and adopts a balancing strategy to recover balance, after which the CoM rapidly converges back to the desired trajectory and resumes accurate tracking. This result illustrates that the proposed controller maintains stable reference tracking under nominal conditions while autonomously prioritizing balance recovery when subjected to large disturbances.

V. CONCLUSION

In this paper, we proposed a method for training humanoid locomotion policies that precisely execute foothold and gait parameter commands. Our approach incorporates the Linear Inverted Pendulum Model to generate online CoM and simple heuristic foot trajectories at policy frequency, enabling accurate swing-foot placement on designated targets while maintaining stable motion. Experiments in simulation and hardware show that the trained policy achieves positional errors $\leq$ 6 cm, allowing the robot to traverse highly constrained terrains such as stepping stones. Beyond accuracy, we also observe improved contact force behavior: The proposed method yields smoother, more bounded vertical ground reaction forces with fewer transition spikes, consistent with more stable contact switching and reduced impact.

These properties are particularly relevant for heavy, high-payload humanoid robots operating in real-world environments with spatially constrained footholds, such as industrial facilities, construction sites, and disaster response scenarios, where precise foot placement and stable contact transitions are critical safety and operational requirements. The modular design of the proposed framework, where the low-level policy can be coupled with any upstream perception and planning system, further makes it amenable to real-world deployment pipelines, as demonstrated by the real-robot experiments using ArUco marker-based online foothold command generation.

Although formal convergence and stability guarantees for the combined framework are theoretically challenging, the LIPM-based preview control provides a physically grounded stability prior through ZMP-based feasibility conditions. The empirical stability of the framework is further supported by smooth and bounded vertical ground reaction forces, successful traversal of stepping stones requiring precise foot placement, and robustness against external disturbances including automatic recovery via high-frequency stepping. These results collectively suggest that the combination of model-based structure and learning-based robustness yields reliable and stable locomotion in practice, while accomplishing this with a simpler control framework than complex model-based approaches.

For future work, we will move beyond a constant height LIPM CoM generator and adopt a centroidal preview–control formulation that produces full 3D CoM trajectories and a resultant wrench, with feasibility enforced through wrench distribution at contacts [40]. This direction enables vertical CoM motion and multi-contact behaviors while retaining real-time performance. We also plan to integrate a high-level footstep planner that, given accurate perception, selects safe and dynamically feasible footholds while rejecting no-step regions and generating robust foothold commands that account for sensing uncertainty, enabling navigation of sparsely accessible terrain by providing the controller with validated foothold sequences. Together, these changes should extend the method beyond flat terrain, improve contact force smoothness, and further enhance the stepping accuracy and stability on heavy humanoids.

ACKNOWLEDGMENT

(Jaeyong Shin and Woohyun Cha contributed equally to this work.)

REFERENCES

[1] I. Radosavovic, S. Kamat, T. Darrell, and J. Malik, "Learning humanoid locomotion over challenging terrain," 2024, *arXiv:2410.03654*.
[2] I. Radosavovic, T. Xiao, B. Zhang, T. Darrell, J. Malik, and K. Sreenath, "Real-world humanoid locomotion with reinforcement learning," *Sci. Robot.*, vol. 9, no. 89, p. 9579, Apr. 2024.
[3] A. Tang, T. Hiraoka, N. Hiraoka, F. Shi, K. Kawaharazuka, K. Kojima, K. Okada, and M. Inaba, "HumanMimic: Learning natural locomotion and transitions for humanoid robot via Wasserstein adversarial imitation," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2024, pp. 13107–13114.
[4] H. Duan, A. Malik, J. Dao, A. Saxena, K. Green, J. Siekmann, A. Fern, and J. Hurst, "Sim-to-real learning of footstep-constrained bipedal dynamic walking," in *Proc. Int. Conf. Robot. Autom. (ICRA)*, May 2022, pp. 10428–10434.
[5] H. Duan, A. Malik, M. S. Gadde, J. Dao, A. Fern, and J. Hurst, "Learning dynamic bipedal walking across stepping stones," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Oct. 2022, pp. 6746–6752.
[6] C. Zhang, W. Xiao, T. He, and G. Shi, "Wococo: Learning whole-body humanoid control with sequential contacts," Tech. Rep., 2024.
[7] C. Zhang, N. Rudin, D. Hoeller, and M. Hutter, "Learning agile locomotion on risky terrains," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Oct. 2024, pp. 11864–11871.
[8] R. Akizhanov, V. Dhédin, M. Khadiv, and I. Laptev, "Learning feasible transitions for efficient contact planning," 2024, *arXiv:2407.11788*.
[9] N. Scianca, D. De Simone, L. Lanari, and G. Oriolo, "MPC for humanoid gait generation: Stability and feasibility," *IEEE Trans. Robot.*, vol. 36, no. 4, pp. 1171–1188, Aug. 2020.

[10] Y. Ding, C. Khazoom, M. Chignoli, and S. Kim, "Orientation-aware model predictive control with footstep adaptation for dynamic humanoid walking," in *Proc. IEEE-RAS 21st Int. Conf. Humanoid Robots (Humanoids)*, Nov. 2022, pp. 299–305.
[11] M. Sombolestan and Q. Nguyen, "Adaptive force-based control of dynamic legged locomotion over uneven terrain," *IEEE Trans. Robot.*, 2024.
[12] S. H. Bang, J. Lee, C. Gonzalez, and L. Sentis, "Variable inertia model predictive control for fast bipedal maneuvers," 2024, *arXiv:2407.16811*.
[13] Z. Zhuang, S. Yao, and H. Zhao, "Humanoid parkour learning," 2024, *arXiv:2406.10759*.
[14] H. Wang, Z. Wang, J. Ren, Q. Ben, T. Huang, W. Zhang, and J. Pang, "BeamDojo: Learning agile humanoid locomotion on sparse footholds," 2025, *arXiv:2502.10363*.
[15] S. Kajita, F. Kanehiro, K. Kaneko, K. Yokoi, and H. Hirukawa, "The 3D linear inverted pendulum mode: A simple modeling for a biped walking pattern generation," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. Expanding Societal Role Robot. Next Millennium*, vol. 1, 2001, pp. 239–246.
[16] Z. Xie, H. Y. Ling, N. H. Kim, and M. van de Panne, "ALLSTEPS: Curriculum-driven learning of stepping stone skills," *Comput. Graph. Forum*, vol. 39, no. 8, pp. 213–224, Dec. 2020.
[17] J. Englsberger, C. Ott, and A. Albu-Schäffer, "Three-dimensional bipedal walking control based on divergent component of motion," *IEEE Trans. Robot.*, vol. 31, no. 2, pp. 355–368, Apr. 2015.
[18] J. Siekmann, Y. Godse, A. Fern, and J. Hurst, "Sim-to-real learning of all common bipedal gaits via periodic reward composition," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2021, pp. 7309–7315.
[19] D. Kim, G. Berseth, M. Schwartz, and J. Park, "Torque-based deep reinforcement learning for task-and-robot agnostic learning on bipedal robots using sim-to-real transfer," *IEEE Robot. Autom. Lett.*, vol. 8, no. 10, pp. 6251–6258, Oct. 2023.
[20] D. Kang, J. Cheng, M. Zamora, F. Zargarbashi, and S. Coros, "RL+ model-based control: Using on-demand optimal control to learn versatile legged locomotion," *IEEE Robot. Autom. Lett.*, vol. 8, no. 10, pp. 6619–6626, Oct. 2023.
[21] K. Green, Y. Godse, J. Dao, R. L. Hatton, A. Fern, and J. Hurst, "Learning spring mass locomotion: Guiding policies with a reduced-order model," *IEEE Robot. Autom. Lett.*, vol. 6, no. 2, pp. 3926–3932, Apr. 2021.
[22] R. Batke, F. Yu, J. Dao, J. Hurst, R. L. Hatton, A. Fern, and K. Green, "Optimizing bipedal maneuvers of single rigid-body models for reinforcement learning," in *Proc. IEEE-RAS 21st Int. Conf. Humanoid Robots (Humanoids)*, Nov. 2022, pp. 714–721.
[23] X. B. Peng, G. Berseth, K. Yin, and M. Van De Panne, "DeepLoco: Dynamic locomotion skills using hierarchical deep reinforcement learning," *ACM Trans. Graph.*, vol. 36, no. 4, pp. 1–13, Aug. 2017.
[24] R. P. Singh, M. Benallegue, M. Morisawa, R. Cisneros, and F. Kanehiro, "Learning bipedal walking on planned footsteps for humanoid robots," in *Proc. IEEE-RAS 21st Int. Conf. Humanoid Robots (Humanoids)*, Nov. 2022, pp. 686–693.
[25] H. J. Lee, S. Hong, and S. Kim, "Integrating model-based footstep planning with model-free reinforcement learning for dynamic legged locomotion," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Oct. 2024, pp. 11248–11255.
[26] F. Jenelten, J. He, F. Farshidian, and M. Hutter, "DTC: Deep tracking control," *Sci. Robot.*, vol. 9, no. 86, p. 5401, Jan. 2024.
[27] S. Gangapurwala, M. Geisert, R. Orsolino, M. Fallon, and I. Havoutis, "RLOC: Terrain-aware legged locomotion using reinforcement learning and optimal control," *IEEE Trans. Robot.*, vol. 38, no. 5, pp. 2908–2927, Oct. 2022.
[28] N. Heess, D. TB, S. Sriram, J. Lemmon, J. Merel, G. Wayne, Y. Tassa, T. Erez, Z. Wang, S. M. A. Eslami, M. Riedmiller, and D. Silver, "Emergence of locomotion behaviours in rich environments," 2017, *arXiv:1707.02286*.
[29] S. Kajita, M. Morisawa, K. Miura, S. Nakaoka, K. Harada, K. Kaneko, F. Kanehiro, and K. Yokoi, "Biped walking stabilization based on linear inverted pendulum tracking," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, Oct. 2010, pp. 4489–4496.
[30] J. Englsberger, G. Mesesan, and C. Ott, "Smooth trajectory generation and push-recovery based on divergent component of motion," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Sep. 2017, pp. 4560–4567.
[31] S. Kajita, F. Kanehiro, K. Kaneko, K. Fujiwara, K. Harada, K. Yokoi, and H. Hirukawa, "Biped walking pattern generation by using preview control of zero-moment point," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2003, pp. 1620–1626.
[32] Z. Fu, A. Kumar, J. Malik, and D. Pathak, "Minimizing energy consumption leads to the emergence of gaits in legged robots," 2021, *arXiv:2111.01674*.
[33] V. Makoviychuk, L. Wawrzyniak, Y. Guo, M. Lu, K. Storey, M. Macklin, D. Hoeller, N. Rudin, A. Allshire, A. Handa, and G. State, "Isaac gym: High performance GPU-based physics simulation for robot learning," 2021, *arXiv:2108.10470*.
[34] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*.
[35] W. Yu, G. Turk, and C. K. Liu, "Learning symmetric and low-energy locomotion," *ACM Trans. Graph.*, vol. 37, no. 4, pp. 1–12, Aug. 2018.
[36] J. R. Cha, W. Cha, J. Shin, D. Kim, and J. Park, "Sim-to-real of humanoid locomotion policies via joint torque space perturbation injection," 2025, *arXiv:2504.06585*.
[37] Z. Chen, X. He, Y.-J. Wang, Q. Liao, Y. Ze, Z. Li, S. S. Sastry, J. Wu, K. Sreenath, S. Gupta, and X. B. Peng, "Learning smooth humanoid locomotion through lipschitz-constrained policies," 2024, *arXiv:2410.11825*.
[38] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, Oct. 2012, pp. 5026–5033.
[39] C. Mastalli, R. Budhiraja, W. Merkt, G. Saurel, B. Hammoud, M. Naveau, J. Carpentier, L. Righetti, S. Vijayakumar, and N. Mansard, "Crocoddyl: An efficient and versatile framework for multi-contact optimal control," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2020, pp. 2536–2542.
[40] M. Murooka, M. Morisawa, and F. Kanehiro, "Centroidal trajectory generation and stabilization based on preview control for humanoid multi-contact motion," *IEEE Robot. Autom. Lett.*, vol. 7, no. 3, pp. 8225–8232, Jul. 2022.

JAEYONG SHIN received the B.S. degree in mechanical engineering from Yonsei University, Seoul, South Korea, in 2021. He is currently pursuing the Ph.D. degree with the Department of Intelligence and Information, Seoul National University, Seoul. His research interests include walking control of a humanoid robot, reinforcement learning, and control system framework.

WOOHYUN CHA received the B.S. degree in mechanical engineering and the M.S. degrees from the Department of Intelligence and Information, Seoul National University, Seoul, South Korea, in 2023 and 2025, respectively. His research interests include walking control of a humanoid robot, reinforcement learning, and control system framework.

JAEHEUNG PARK (Senior Member, IEEE) received the B.S. and M.S. degrees in aerospace engineering from Seoul National University, South Korea, in 1995 and 1999, respectively, and the Ph.D. degree in aeronautics and astronautics from Stanford University, Stanford, CA, USA, in 2006. From 2006 to 2009, he was a Post-doctoral Researcher and a Research Associate with Stanford Artificial Intelligence Laboratory. From 2007 to 2008, he worked part-time with Hansen Medical Inc., a medical robotics company, USA. Since 2009, he has been a Professor with the Department of Intelligent Convergence Systems, Seoul National University. His research interests include robot-environment interaction, contact force control, robust haptic teleoperation, multicontact control, whole-body dynamic control, biomechanics, and medical robotics.